2 Years Experience | Playwright | TypeScript | Automation Testing

Mock Interview Guide — detailed, clear, interview-ready answers, examples, follow-ups, and memory hooks

PART 1 — PROJECT & QA

1. Tell me about yourself and explain your current project, application, and Playwright responsibilities.

Candidate Answer:

I have around 2 years of experience in QA automation, mainly working with Playwright and TypeScript.

In my current project, I work on web application testing where my responsibilities include understanding requirements, preparing test scenarios, executing manual testing, identifying defects, and automating stable and repetitive scenarios using Playwright.

For automation, I follow the Page Object Model and use Playwright's built-in locators, assertions, fixtures, and configuration features.

I also work with test data, authentication, screenshots, traces, and reports. I run tests locally as well as through CI, analyze failures, and maintain the automation suite when the application changes.

Apart from UI automation, I understand API validation and how API-based setup can be used to make UI tests faster.

Real example:
For a new purchase or login feature, I first understand the acceptance criteria, manually validate the important scenarios, identify regression scenarios, and then automate the stable flows using Playwright.

Interviewer Follow-up: What exactly do you automate?
I prioritize business-critical, repetitive, regression-heavy, and stable scenarios. I generally avoid automating scenarios that are executed only once, are highly unstable, or require subjective visual validation unless automation provides clear value.

Memory: Requirement → Manual → Scenarios → Automation → Execute → Analyze → Maintain

2. Explain your testing process for a new feature from requirement analysis through automation and execution.

Candidate Answer:

When I receive a new feature, I start by understanding the requirement and acceptance criteria. I identify the positive, negative, boundary, validation, and integration scenarios.

Then I clarify any ambiguous requirements with the BA, product owner, or developer.

I prepare test cases and execute the important scenarios manually first. Once I understand the feature and the flow is stable, I identify scenarios suitable for automation.

I implement the automation using our framework, execute it locally, review failures, and then integrate it into regression or CI execution.

Finally, I report defects, verify fixes, perform regression testing, and maintain the automation if the feature changes.

Interviewer Follow-up: Do you automate before manual testing?
Usually I prefer to understand and validate the feature first. That helps me understand the actual behavior and avoid automating incorrect assumptions.

Memory: Understand → Design → Manual → Automate → Execute → Defect → Regression

3. How do you decide which scenarios should be automated versus manual?

Candidate Answer:

I consider business value, repetition, stability, execution frequency, and maintenance cost.

I prioritize critical business flows, regression scenarios, repetitive validations, data-driven scenarios, and scenarios that need to run across multiple browsers or environments.

I normally keep exploratory testing, one-time scenarios, and highly subjective visual checks manual unless there is a strong reason to automate.

Interviewer Follow-up: Would you automate every regression test?
Not necessarily. Automation should provide long-term value. If a test is extremely unstable or expensive to maintain compared to its execution value, I would evaluate whether automation is justified.

Memory: Value + Repeatability + Stability + Maintenance

4. Explain Severity vs Priority with a real example.

Candidate Answer:

Severity represents the impact of a defect on the application, while priority represents how urgently the defect should be fixed.

High Severity + High Priority:
The application crashes when a user submits a payment.

Low Severity + High Priority:
A spelling mistake in the company's homepage heading before a major release.

Interviewer Follow-up: Can a low-severity bug have high priority?
Yes. Priority depends on business urgency, release requirements, customer visibility, and business impact, not only technical severity.

Memory: Severity = Impact | Priority = Urgency

5. How do you handle an unclear requirement or a developer disputing a defect?

Candidate Answer:

First, I don't treat it as an argument. I go back to the requirement, acceptance criteria, design, or business expectation.

I reproduce the issue with clear steps and provide evidence such as screenshots, videos, logs, network information, or traces.

If the requirement itself is unclear, I involve the appropriate BA, product owner, or business stakeholder to clarify the expected behavior.

If the requirement confirms that my observation is a defect, I explain the evidence to the developer. If the expected behavior is different, I update my test case accordingly.

Strong sentence:
I focus on evidence and expected behavior rather than opinion.

Memory: Requirement → Reproduce → Evidence → Discuss → Clarify

PART 2 — TYPESCRIPT

6. Why use TypeScript instead of JavaScript for test automation?

Candidate Answer:

TypeScript is a superset of JavaScript that adds static typing and other language features.

In automation frameworks, TypeScript helps catch many errors during development instead of discovering them only at runtime.

It also improves code readability, IDE autocomplete, refactoring, and maintainability, which becomes especially useful when the framework grows with Page Objects, fixtures, utilities, and test data models.

Example:

function login(username: string, password: string): void {
// ...
}

Interviewer Follow-up: Does TypeScript run directly in the browser?

TypeScript is transpiled into JavaScript, and the resulting JavaScript is executed.

Memory: Types → Errors earlier → IDE → Maintainability

7. Explain let, const, and var. Which do you normally use?

Candidate Answer:

var, let, and const are used for variable declarations, but their scoping and reassignment behavior differ.

var is function-scoped and can be redeclared, so I generally avoid it in modern TypeScript.

let is block-scoped and can be reassigned.

const is block-scoped and cannot be reassigned.

In automation code, I normally use const by default and use let only when the value needs to change.

Example:

```
const baseUrl = "https://example.com";

let retryCount = 0;
retryCount++;

var oldStyle = "avoid";
```

Memory: const = default | let = changes | var = generally avoid

8. Explain interface, type, class, access modifiers, and inheritance with automation examples.

Candidate Answer:

An interface primarily defines the structure or contract of an object or class.

A type can also describe object structures, but it can additionally represent unions, intersections, primitives, and other type compositions.

A class is used to create objects containing data and behavior.

Access modifiers control visibility. public is accessible generally, private is restricted to the class, and protected is available to the class and its subclasses.

Inheritance allows a class to reuse or extend behavior from another class.

Automation example:

```
interface LoginData {
    username: string;
    password: string;
}

class LoginPage {
    constructor(private page: Page) {}

    async login(data: LoginData) {
        await this.page.getByLabel("Username").fill(data.username);
        await this.page.getByLabel("Password").fill(data.password);
    }
}
```

Interviewer Follow-up: Why use interfaces for test data?

They provide a clear contract for the expected structure of test data and allow TypeScript to catch incorrect data shapes during development.

Memory: Interface = contract | Type = type composition | Class = behavior + data | private = class only

9. Explain async/await and Promises. Why are they important in Playwright?

Candidate Answer:

A Promise represents the eventual result of an asynchronous operation.

async makes a function return a Promise, and await allows us to wait for the Promise to resolve before continuing.

Playwright APIs are asynchronous, so we normally use async/await to ensure actions, navigation, and assertions complete in the expected sequence.

Example:

```
await page.getByRole("button", { name: "Login" }).click();

await expect(page.getByText("Dashboard")).toBeVisible();
```

Important example:

```
const [popup] = await Promise.all([
    page.waitForEvent("popup"),
    page.getByText("Open Report").click()
]);
```

Why Promise.all?

Because we start waiting for the popup before performing the action that creates it.

Interviewer Follow-up: What happens if you forget await?
The operation may still be pending while the next statement executes, which can cause race conditions, unexpected behavior, or incorrect assertions.

Memory: Promise = future result | await = wait for result

10. How do you handle errors using try/catch and useful error messages?

Candidate Answer:

I use try/catch when I need to handle an expected failure, add meaningful context, perform cleanup, or convert a low-level error into a useful framework error.

I avoid wrapping every Playwright statement in try/catch because Playwright and Test Runner already provide useful failure information.

Example:

```
try {
    await loginPage.login(username, password);
} catch (error) {
    throw new Error(`Login failed for user: ${username}`);
}
```

Interviewer Follow-up: Why not catch (error) { }?

Because silently swallowing the exception hides the actual failure and makes debugging difficult.

Memory: Catch → Add context → Don't hide failure

PART 3 — PLAYWRIGHT

11. Explain your Playwright framework architecture.

Candidate Answer:

Our Playwright framework follows a layered structure.

The configuration manages browsers, projects, environments, retries, workers, reporters, and baseURL.

Page Objects contain page-specific locators and reusable actions.

Fixtures provide reusable setup and dependencies.

Tests contain the actual business scenarios and assertions.

Test data is maintained separately from test logic.

Utilities contain generic reusable functions.

Reports, traces, screenshots, and other artifacts help us analyze failures.

CI executes the suite automatically.

Architecture:

```
playwright.config.ts
        ↓
```

```
Fixtures
        ↓
Page Objects
        ↓
Tests
        ↓
Test Data / Utilities
        ↓
Reports / Artifacts
        ↓
CI
```

Interviewer Follow-up: Should assertions be inside Page Objects?

I prefer business-level assertions in the test because the test should describe what is being validated. Page Objects should primarily expose page behavior and reusable interactions. Small component-level validation helpers can be reasonable when they improve reuse and readability.

Memory: Config → Fixtures → POM → Tests → Data → Reports → CI

12. Compare getByRole(), getByText(), getByTestId(), CSS, and XPath locators.

Candidate Answer:

I prefer user-facing and semantic locators first.

getByRole() identifies elements based on their accessible role and name.

getByLabel() is useful for form controls associated with labels.

getByText() targets visible text.

getByTestId() is useful when the application provides a stable test-specific attribute.

CSS and XPath are useful when semantic or test-specific locators aren't suitable, but they can be more coupled to implementation details.

Typical preference:
getByRole() → getByLabel() → getByText() → getByTestId() → CSS/XPath.

Don't blindly follow this order; choose the most stable locator for the element.

Interviewer Follow-up: Why is XPath not your first choice?
It can become tightly coupled to DOM structure and can be harder to maintain when the UI changes.

13. Why are Playwright locators more reliable than direct element handles?

Candidate Answer:

Playwright locators are designed for repeated interaction with elements and provide built-in waiting and retry behavior.

A locator represents a way to find an element rather than immediately capturing a particular DOM element.

This makes locators more resilient when the DOM changes between operations.

Example:

```
const loginButton = page.getByRole("button", { name: "Login" });

await loginButton.click();
await expect(loginButton).toBeVisible();
```

Interviewer Follow-up: What does locator strictness mean?

If an operation expects a single element but the locator matches multiple elements, Playwright can fail with a strict mode violation rather than silently selecting an arbitrary element.

Memory: Locator = find + wait + retry

14. Explain Playwright auto-waiting. When would you use an explicit wait?

Candidate Answer:

Playwright automatically waits for actionability before performing actions.

For example, before clicking, it checks whether the element is visible, stable, able to receive events, and enabled.

This reduces the need for hard-coded sleeps.

Use condition-based waits when synchronization is needed:

```
await page.waitForURL("**/dashboard");

await page.waitForResponse(response =>
    response.url().includes("/api/orders")
);
```

Avoid arbitrary sleeps such as waitForTimeout(5000) unless there is a specific debugging or exceptional reason.

Memory: Don't wait for time → wait for condition

15. How do you handle dynamic elements without hard-coded waits?

Candidate Answer:

I avoid waiting for an arbitrary amount of time. Instead, I use resilient locators and Playwright's built-in auto-waiting.

If I need to synchronize with application behavior, I wait for a meaningful condition such as an element state, URL, API response, or specific page state.

Example:

```
await expect(page.getByText("Order Created")).toBeVisible();

Better than:
await page.waitForTimeout(3000);

Interviewer Follow-up: What if the element appears after an API call?
I can wait for the relevant response or the resulting UI state, depending on what the test is actually validating.
```

16. How do you handle dropdowns, checkboxes, dialogs, frames, and file uploads?

Native dropdown:

```
await page.locator("select").selectOption("india");
```

Custom dropdown:

```
await page.getByRole("combobox").click();
await page.getByRole("option", { name: "India" }).click();
```

Checkbox:

```
await page.getByRole("checkbox", { name: "Remember me" }).check();
```

Dialog:

```
page.on("dialog", async dialog => {
    await dialog.accept();
```

```
});
```

Frame:

```
const frame = page.frameLocator("#payment-frame");

await frame.getByLabel("Card Number").fill("1234");
```

Upload:

```
await page.setInputFiles("input[type=file]", "test.pdf");
```

Interviewer Follow-up: Why frameLocator()?

It allows me to locate and interact with elements inside an iframe without manually switching browser context like Selenium.

Memory: Identify control type → use its Playwright-native API

17. How do you handle multiple tabs/pages and popups?

Candidate Answer:

In Playwright, a browser context can contain multiple pages. For a popup created by an action, I listen for the popup event and perform the action in the same synchronization block.

Example:

```
const [popup] = await Promise.all([
    page.waitForEvent("popup"),
    page.getByRole("link", { name: "Open Report" }).click()
]);

await popup.waitForLoadState();
await expect(popup).toHaveTitle(/Report/);
```

Interviewer Follow-up: Why Promise.all?

Because the event listener needs to be registered before the click triggers the popup.

18. How do you capture screenshots, videos, traces, and logs on failure?

Candidate Answer:

I configure Playwright to capture artifacts based on our debugging and CI requirements. Screenshots provide visual evidence, videos can show the sequence of actions, and traces provide detailed execution information including actions, DOM snapshots, network information, and timing.

Typical config:

use: {
screenshot: "only-on-failure",
trace: "on-first-retry",
video: "retain-on-failure"
}

Interviewer Follow-up: Which one do you use most for debugging?

Trace Viewer is especially useful because it gives a timeline of the test execution and allows me to inspect actions and page state around the failure.

19. Explain browser, browser context, and page.

Candidate Answer:

Browser represents the browser instance.

BrowserContext represents an isolated browser session, similar to an incognito session, with its own cookies and storage.

Page represents a tab inside a browser context.

Diagram:

```
Browser
 ■■■ Context 1
 ■    ■■■ Page
 ■    ■■■ Page
 ■
 ■■■ Context 2
      ■■■ Page
```

Strong interview statement:
Contexts provide test isolation without requiring a separate browser process for every test.

Memory: Browser → Context → Page

20. How do you manage authentication using storageState?

Candidate Answer:

Instead of performing the login UI flow before every test, I can authenticate once and save the authenticated browser state using storageState.

That state can then be reused by tests that require an authenticated session.

Example:

```
await context.storageState({
    path: "auth.json"
});
```

Then:
use: {
storageState: "auth.json"
}

Important:

Don't use this blindly for login tests. If the test validates login, perform the login. If it validates an authenticated business flow, reuse the state.

Memory: Login test → UI login | Other authenticated tests → storageState

21. How do you parameterize tests and manage test data?

Candidate Answer:

I keep test data separate from test logic. Depending on the scenario, I can use JSON, CSV, environment variables, fixtures, or generated data.

For data-driven testing, I create separate test cases for each dataset when individual reporting is important.

I also generate unique values for fields that require uniqueness, especially when tests run in parallel.

Example:

```
const users = [
```

```
    { username: "user1", password: "pass1" },
    { username: "user2", password: "pass2" }
];

for (const user of users) {
    test(`login - ${user.username}`, async ({ page }) => {
        // ...
    });
}
```

Interviewer Follow-up: Why not put the loop inside one test?

Then the runner sees it as one test, so failures for individual datasets are less isolated and reporting becomes less granular.

22. How do you organize smoke, regression, and critical tests?

Candidate Answer:

Smoke testing is a smaller set of high-level tests used to verify that the build is stable enough for deeper testing.

Regression testing covers a broader set of existing functionality to identify unintended impact from changes.

Critical tests focus on high-business-impact flows that must work for the release.

Example:

```
Smoke
→ Login
→ Dashboard
→ Create transaction

Regression
→ Login
→ Users
→ Reports
→ Transactions
→ Settings
→ Notifications
...
```

Playwright tags:

test("@smoke login", ...)
test("@regression create order", ...)

Run:
npx playwright test --grep @smoke

23. A test passes locally but fails intermittently in CI. How do you debug it?

Candidate Answer:

I would first determine whether the issue is related to timing, environment, test data, browser differences, parallel execution, or an actual application defect.

I would inspect the CI logs, trace, screenshot, video, and failure details.

Then I would reproduce it under CI-like conditions by using the same browser, environment variables, workers, and test data.

I would avoid simply increasing retries because retries can hide the underlying problem.

Debugging order:

```
CI logs
 ↓
Trace
 ↓
Screenshot/video
 ↓
Locator
 ↓
Timing
 ↓
Test data
 ↓
Environment
 ↓
Parallelism
 ↓
Browser/version
```

Strong sentence:

My goal is to identify the root cause of the flakiness rather than make the failure less visible.

PART 4 — FRAMEWORK & CI/CD

24. Explain your Page Object Model design. What belongs and does not belong in a Page Object?

Candidate Answer:

A Page Object should contain page-specific locators and reusable interactions representing the behavior of that page.

It should not contain test-specific data, test orchestration, unrelated business workflows, or large amounts of assertion logic.

Good:

```
class LoginPage {
    constructor(private page: Page) {}

    async login(username: string, password: string) {
        await this.page.getByLabel("Username").fill(username);
        await this.page.getByLabel("Password").fill(password);
        await this.page.getByRole("button", { name: "Login" }).click();
    }
}
```

Test:

```
test("valid login", async ({ page }) => {
    const loginPage = new LoginPage(page);

    await loginPage.login("user", "password");

    await expect(page.getByText("Dashboard")).toBeVisible();
});
```

Memory: POM = page behavior | Test = scenario + validation

25. How do you configure playwright.config.ts for browsers, environments, retries, workers, reporters, and baseURL?

Candidate Answer:

playwright.config.ts is the central configuration file for the test framework. I use it to configure the test directory, baseURL, projects, browsers, retries, workers, reporters, timeout settings, and shared browser context options.

Example:

```
export default defineConfig({
    testDir: "./tests",

    use: {
        baseURL: process.env.BASE_URL,
        screenshot: "only-on-failure",
        trace: "on-first-retry"
    },

    retries: process.env.CI ? 2 : 0,

    reporter: [
        ["html"],
        ["junit", { outputFile: "results.xml" }]
    ]
});
```

Interviewer Follow-up: Why different retries locally and CI?

CI environments can be more variable, so a limited retry can help collect useful failure information. Locally, I prefer failures to surface immediately so I can debug the root cause.

26. How do you run tests in parallel and prevent shared-data conflicts?

Candidate Answer:

Playwright supports parallel execution through workers. Parallel execution improves speed, but tests must be isolated.

I avoid sharing mutable state between tests and generate unique test data where required.

For example, if multiple tests create users, each test should use a unique email instead of using the same user.

Example:

```
const email = `test-${crypto.randomUUID()}@example.com`;
```

Interviewer Follow-up: What else can cause parallel conflicts?

Shared database records, static files, global variables, ports, shared accounts, and tests that depend on execution order.

Memory: Parallel = isolation + unique data + no dependencies

27. How do you integrate Playwright with Jenkins or GitHub CI and publish reports?

Candidate Answer:

I integrate Playwright into CI by installing dependencies, installing the required browsers, configuring environment variables, executing the tests, and publishing reports and failure artifacts.

Typical flow:

Checkout
↓
Install Node
↓
npm ci
↓
npx playwright install
↓
Set environment variables
↓
Run tests

↓
Generate report
↓
Publish artifacts

Commands:
npm ci
npx playwright install --with-deps
npx playwright test

Interviewer Follow-up: What artifacts would you publish?

HTML reports, JUnit results if required by the CI system, screenshots, videos, and traces for failed tests.

PART 5 — REAL-TIME SCENARIOS

28. Your suite has 300 tests and 30 are flaky in CI. How do you find root causes instead of just increasing retries?

Candidate Answer:

I would first classify the failures and identify patterns rather than treating all 30 failures as the same problem.

I would analyze traces, screenshots, videos, logs, and failure messages.

Then I would group failures by root cause—for example, timing issues, locator instability, test-data conflicts, environment problems, network issues, or application defects.

I would reproduce representative failures and fix the underlying causes.

I would use retries only as a limited safety mechanism after understanding the root cause, not as the primary solution.

Example classification:

```
30 failures

10 → shared test data
8  → unstable locator
5  → API timeout
4  → environment
2  → real application bugs
1  → infrastructure
```

Strong sentence:

I would reduce 30 symptoms into a smaller number of root causes.

29. A login API is slow and UI tests take too long. How can APIRequestContext improve the test suite?

Candidate Answer:

If every UI test performs login through the UI, it can significantly increase execution time.

If the test scenario isn't specifically validating the login UI, I can use APIRequestContext to authenticate or create the required state through an API.

Then I can use the UI only for the behavior I actually want to validate.

Concept:

```
API
 ↓
Create/authenticate test user
 ↓
Prepare state
 ↓
UI
 ↓
Test actual business flow
```

Why?

Instead of repeating UI login setup across hundreds of tests, move authentication/setup outside the repeated UI flow where appropriate.

Important:
I would still keep dedicated UI login tests because the login functionality itself needs to be validated.

30. Your application has changing CSS classes and dynamic IDs. How do you create stable locators?

Candidate Answer:

I avoid relying on dynamic implementation details whenever possible.

I first look for accessible locators such as role and accessible name, labels, or stable test IDs.

If the application doesn't expose reliable attributes, I would discuss adding a dedicated test ID with developers rather than creating extremely fragile XPath expressions.

Example:

```
await page.getByRole("button", { name: "Submit" }).click();
```

```
Or:
await page.getByTestId("submit-button").click();
```

Interviewer Follow-up: What if there are 5 Submit buttons?

I would narrow the locator using the appropriate parent, container, or filtering strategy rather than blindly using .first().

Example:

```
page
.getByRole("row")
.filter({ hasText: "Jashu" })
.getByRole("button", { name: "Edit" });
```

Memory: Semantic → Test ID → CSS/XPath

31. Release is tomorrow and 40 of 300 regression tests fail. How do you triage and communicate release risk?

Candidate Answer:

First, I would not treat all 40 failures equally. I would classify them based on business impact, severity, affected functionality, failure consistency, and whether they are genuine product defects or automation/environment issues.

I would identify which failures affect critical business flows and which are flaky or infrastructure-related.

I would communicate the results clearly to the QA lead, development team, and product stakeholders, including the number of genuine defects, affected areas, severity, and evidence.

Then the release decision would be made based on the team's release criteria and business risk.

Example:

```
40 failures

15 → automation/environment issues
10 → known flaky tests
8  → low-impact defects
5  → critical business-flow defects
2  → new high-impact defects
```

Communication example:

The regression suite has 40 failures, but after triage, 25 are related to automation/environment or known issues. 15 require product attention, including failures in critical business flows. I've attached the evidence and impact analysis for the release decision.

Strong interview sentence:
I would provide the evidence and risk assessment; the release decision should be based on the team's release criteria and stakeholders.

■ YOUR 31-QUESTION MEMORY MAP

Don't memorize 31 answers separately. Memorize these 8 blocks:

```
1–5   → QA thinking
6–10  → TypeScript
11–20 → Playwright
21–23 → Test data + execution
24–27 → Framework + CI
28–31 → Real-world SDET scenarios
```

Interview pattern

```
QUESTION
   ↓
DEFINE
   ↓
HOW I DO IT
   ↓
REAL EXAMPLE
   ↓
WHY
   ↓
EDGE CASE
   ↓
FOLLOW-UP
```

Most important questions to master first

Tier 1: 1, 9, 11, 12, 13, 14, 19, 20, 24, 25, 26, 28, 29, 30, 31.

Tier 2: 2, 3, 5, 6, 7, 8, 10, 15, 16, 17, 18, 21, 22, 23, 27.

Final preparation advice

Don't memorize these answers word-for-word. Memorize the structure and keywords, then speak naturally. For example, for auto-waiting, remember: Actionability → visible → stable → receives events → enabled → no hard-coded sleep → condition-based wait when necessary. Then explain it in your own words.